Hospital Management System - V - 2
Project Report

Student Details

Name: Radha N
Roll number: 23f1001297
Email: 23f1001297@ds.study.iitm.ac.in
About me:
I am a student in the IIT Madras BS Degree in Data Science program. I am interested in web development and backend systems using python and flask . I enjoy building applications that solve real-world problems and improve efficiency in healthcare and service management.

Project Details

Project Title: Hospital Management System V - 2
Problem Statement: Hospitals often struggle to efficiently manage patient appointments, doctor availability and appointment records. A centralized system is required where patients can book appointments, doctors can manage schedules, and automated remainders help reduce missed appointments.
Approach: The system was developed as a full-stack web application using Flask for backend and Vue.js for frontend. Patients can register, book appointments and track their visit history , while doctors can manage their schedules and complete appointments. Celery and Redis are used for asynchronous background tasks such as sending daily appointment reminders and generating monthly doctor reports.

AI / LLM Declaration

I used ChatGPT to assist with debugging Celery background tasks, improving code structure , and generating documentation content.

The estimated AI usage is approximately 10-15%, mainly for debugging assistance and explanation for backend concepts.

All major design decisions, implementation, testing and integration were completely manual.

Technologies and Frameworks used

Technology	Purpose
Flask	Backend web framework
SQLite	Database for sorting users, doctors and appointments
SQLAlchemy	ORM for database operations
Vue.js	Frontend framework
Redis	Message broker for Celery
Celery	Background task processing
Flask-CORS	Handling cross-origin requests
Flask-Caching	Performance optimization
SMTP(gmail)	Sending email remainders

Database Scheme / E R Diagram

Tables

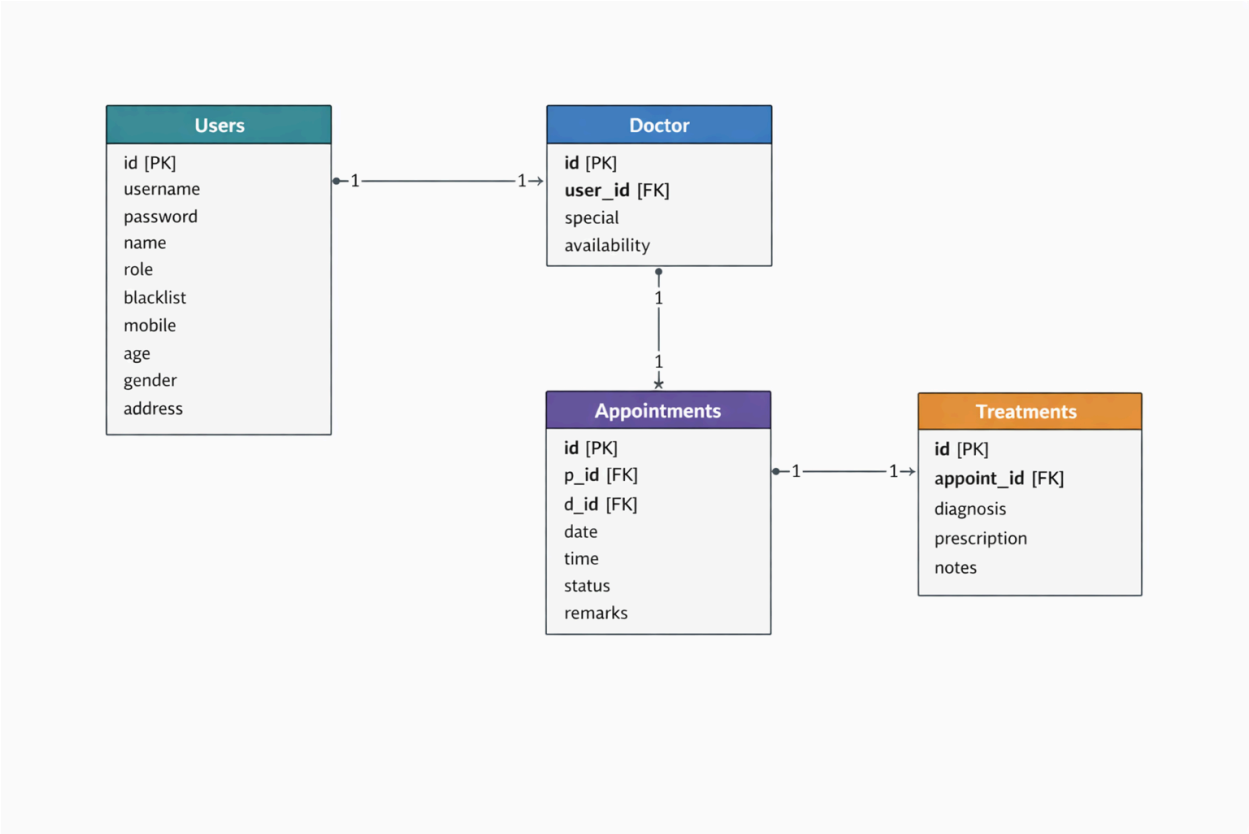
1. Users - stores user account and personal details (id, username, password, name, role, blacklist, mobile, age, gender, address)
2. Doctor - stores doctor-specific information (id, user_id, special, availability)
3. Appointments - stores appointment details between patient and doctor (is, p_id, d_id, date, time, status, remarks)
4. Treatments - stores treatment details for completed appointments (id, appoint_id, diagnosis, prescription, notes)

Relationships

One to One - Users -> Doctor
A doctor record is linked to user account
One to Many
Users -> Appointments(patient) one patient can book multiple appointments.
Doctor -> Appointments one doctor can have multiple appointments.

Appointments -> Treatments each appointment can have a treatment record after completion.

E R Diagram



API Resource Endpoints

Endpoint	Method	Description
/login	POST	Authenticate user and return user details
/register	POST	Register a new patient account
/profile/<user_id>	GET	Retrieve profile details of user
/profile/update/<user_id>	PUT	Update patient profile information

/docs	GET	Fetch list of doctors with specialization
/patient/docs	GET	Fetch doctor available for patient booking
/patient/book	POST	Book appointment with doctor
/patient/appo/<patient_id>	GET	Retrieve all appointment for patient
/patient/cancels/<appt_id>	PUT	Cancel booked appointment
/doctor/appointments/<doctor_id>	GET	Fetch all appointment assigned to doctor
/doctor/complete/<appt_id>	POST	Mark an appointment as completed and add treatment details
/doctor/cancel/<appt_id>	PUT	Cancel appointment by doctor
/appoints	GET	Retrieve all appointment in system
/a_stats	GET	Return statistics about doctors, patient and appointment
/search	GET	Search for doctor or patient name
/daily	GET	Return daily appointment schedule summary
/monthly	GET	Return monthly doctor appointment analytics

Architecture and Features

Architecture overview

The system follows a modular Flask backend architecture with asynchronous task processing using Celery.

- app.py – Main Flask application that initializes the API, database connection, caching system, and registers Blueprints.

- database.py – Defines SQLAlchemy database models such as Users, Doctors, Appointments, and Treatments.

- doctor.py – Contains API routes related to doctor operations like viewing appointments, completing consultations, and accessing patient history.

- patient.py – Contains API routes for patient functionalities such as booking appointments, viewing appointment history, and exporting records.

- tasks.py – Defines background tasks executed by Celery, such as sending appointment reminders, generating monthly doctor reports, and exporting patient data.

- celery_worker.py – Configures the Celery worker, Redis broker, and scheduled background tasks.

- Redis – Used as a message broker for Celery and also for caching frequently accessed API responses.

- SQLite (hms.db) – Database used for storing users, doctors, appointments, and treatment records.

Implemented features

- User authentication system for Admin, Doctor, and Patient roles
- Patient registration and login
- Doctor management (add, edit, delete doctors)
- Appointment booking, cancellation, and completion
- Patient medical history tracking
- Doctor treatment record management
- Search functionality for doctors and patients
- System statistics API for monitoring appointments and users

- Daily appointment summary API
- Monthly doctor report generation
- CSV export of patient treatment history
- Email reminders for scheduled appointments

Additional features

- Background task processing using Celery
- Scheduled jobs using Celery Beat for daily reminders and monthly reports
- Redis caching to improve performance of frequently accessed APIs
- Automated HTML report generation for doctors

Video Presentation

Drive link:
https://drive.google.com/file/d/1LU_JlDDI1WXE_4EMUlHd5MPEBrqov4XS/view?usp=sharing